Computer Vision

LABSHEET - 3

Name-Harsh Raj

Cu id- 240251762

#Question 1

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
import time

# Load image
image_path = "image.jpg"

# Read image in color
image = cv2.imread(image_path)

if image is None:
    raise FileNotFoundError("Image not found. Check the image path.")

# Convert BGR to RGB for displaying with Matplotlib
image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

# Convert to grayscale
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
```

```
print("Image shape:", image.shape)
print("Grayscale shape:", gray.shape)

#question 2

plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
plt.imshow(image_rgb)
plt.title("Original Color Image")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(gray, cmap="gray")
plt.title("Original Grayscale Image")
plt.axis("off")

plt.tight_layout()
plt.show()

#question 3

gaussian = cv2.GaussianBlur(gray, (5, 5), 0)

plt.figure(figsize=(6, 5))
```

```
plt.imshow(gaussian, cmap="gray")
plt.title("Gaussian Blur")
plt.axis("off")
plt.show()

#Question 4

median = cv2.medianBlur(gray, 5)

plt.figure(figsize=(6, 5))
plt.imshow(median, cmap="gray")
plt.title("Median Filter")
plt.axis("off")
plt.show()

#Question 5

mean = cv2.blur(gray, (5, 5))

plt.figure(figsize=(6, 5))
plt.imshow(mean, cmap="gray")
plt.title("Average / Mean Filter")
plt.axis("off")
plt.show()

 #question 6
```

```
laplacian = cv2.Laplacian(gray, cv2.CV_64F)

# Convert to displayable 8-bit image
laplacian_abs = cv2.convertScaleAbs(laplacian)

plt.figure(figsize=(6, 5))
plt.imshow(laplacian_abs, cmap="gray")
plt.title("Laplacian Filtering")
plt.axis("off")
plt.show()

#question 7
sobel_x = cv2.Sobel(gray, cv2.CV_64F, 1, 0, ksize=3)
sobel_x = cv2.convertScaleAbs(sobel_x)

sobel_y = cv2.Sobel(gray, cv2.CV_64F, 0, 1, ksize=3)
sobel_y = cv2.convertScaleAbs(sobel_y)

sobel_combined = cv2.magnitude(
    cv2.Sobel(gray, cv2.CV_32F, 1, 0, ksize=3),
    cv2.Sobel(gray, cv2.CV_32F, 0, 1, ksize=3)
)

sobel_combined = cv2.convertScaleAbs(sobel_combined)
```

```
plt.figure(figsize=(15, 5))

plt.subplot(1, 3, 1)
plt.imshow(sobel_x, cmap="gray")
plt.title("Sobel X - Horizontal Edges")
plt.axis("off")

plt.subplot(1, 3, 2)
plt.imshow(sobel_y, cmap="gray")
plt.title("Sobel Y - Vertical Edges")
plt.axis("off")

plt.subplot(1, 3, 3)
plt.imshow(sobel_combined, cmap="gray")
plt.title("Combined Sobel")
plt.axis("off")

plt.tight_layout()
plt.show()

#question 8
print("\n==========================================")
print("QUESTION 8: FILTER COMPARISON")
print("==========================================")

print("\nLOW-PASS FILTERS")
```

```
print("-----------------------------------------------")
print("Mean Filter")
print("  Noise Reduction : Good")
print("  Edge Preservation: Low")
print(f"  Time            : {mean_time:.6f} seconds")

print("\nGaussian Filter")
print("  Noise Reduction : Good")
print("  Edge Preservation: Medium")
print(f"  Time            : {gaussian_time:.6f} seconds")

print("\nMedian Filter")
print("  Noise Reduction : Very Good for impulse noise")
print("  Edge Preservation: Very Good")
print(f"  Time            : {median_time:.6f} seconds")

print("\nHIGH-PASS FILTERS")
print("-----------------------------------------------")

print("\nLaplacian Filter")
print("  Noise Reduction : Poor")
print("  Edge Preservation: Excellent")
print(f"  Time            : {laplacian_time:.6f} seconds")

print("\nSobel Filter")
print("  Noise Reduction : Poor")
```

```
print("  Edge Preservation: Very Good")
print(f"  Time            : {sobel_time:.6f} seconds")

print("\nCanny Edge Detection")
print("  Noise Reduction : Good")
print("  Edge Preservation: Excellent")
print(f"  Time            : {canny_time:.6f} seconds")

# ----------------------------------------------------------
# QUESTION 9
# Display Original and Filtered Images
# ----------------------------------------------------------

plt.figure(figsize=(15, 10))

# Original
plt.subplot(2, 4, 1)
plt.imshow(image_rgb)
plt.title("Original Image")
plt.axis("off")

# Grayscale
plt.subplot(2, 4, 2)
plt.imshow(gray, cmap="gray")
plt.title("Grayscale")
plt.axis("off")
```

```
# Mean
plt.subplot(2, 4, 3)
plt.imshow(mean_filter, cmap="gray")
plt.title("Mean Filter")
plt.axis("off")

# Gaussian
plt.subplot(2, 4, 4)
plt.imshow(gaussian_filter, cmap="gray")
plt.title("Gaussian Filter")
plt.axis("off")

# Median
plt.subplot(2, 4, 5)
plt.imshow(median_filter, cmap="gray")
plt.title("Median Filter")
plt.axis("off")

# Laplacian
plt.subplot(2, 4, 6)
plt.imshow(laplacian, cmap="gray")
plt.title("Laplacian Filter")
plt.axis("off")

# Sobel
```

```
plt.subplot(2, 4, 7)
plt.imshow(sobel, cmap="gray")
plt.title("Sobel Filter")
plt.axis("off")

# Canny
plt.subplot(2, 4, 8)
plt.imshow(canny, cmap="gray")
plt.title("Canny Edge Detection")
plt.axis("off")

plt.suptitle("Comparison of Low-Pass and High-Pass Filters", fontsize=16)

plt.tight_layout()
plt.show()

# ----------------------------------------------------
# QUESTION 9
# Observations
# ----------------------------------------------------

print("\n==========================================")
print("QUESTION 9: OBSERVATIONS")
print("==========================================")

print("""
```

1. Mean Filter:

Strength:

- Simple and fast.
- Reduces image noise.

Limitation:

- Blurs edges and fine details.

2. Gaussian Filter:

Strength:

- Provides smooth noise reduction.
- Produces natural-looking results.

Limitation:

- Can blur fine image details.

3. Median Filter:

Strength:

- Very effective for salt-and-pepper noise.
- Preserves edges better.

Limitation:

- More computationally expensive than mean filtering.

4. Laplacian Filter:

Strength:

- Strongly detects edges and fine details.

- Fast computation.

Limitation:

- Can amplify noise.

5. Sobel Filter:

Strength:

- Detects horizontal and vertical edges.

- Computationally efficient.

Limitation:

- Sensitive to noise.

- Does not produce as thin edges as Canny.

6. Canny:

Strength:

- Produces clear and thin edges.

- Good edge detection performance.

Limitation:

- More computationally expensive.

- Requires threshold selection.

""")

```
# QUESTION 10
# Interpretation and Real-World Applications
# ----------------------------------------------------

print("\n==============================================")
print("QUESTION 10: INTERPRETATION")
print("==============================================")

print("""
Low-pass filters remove high-frequency components from an image.
Therefore, they are mainly used for noise reduction and smoothing.

High-pass filters emphasize high-frequency components.
Therefore, they are mainly used for edge and detail detection.

Real-world applications of spatial filtering include:

1. Medical Imaging
  - Noise reduction in X-ray, CT and MRI images.
  - Detection of boundaries and structures.

2. Face Recognition
  - Image smoothing and feature extraction.

3. Autonomous Vehicles
  - Detection of road boundaries, vehicles and pedestrians.
```

4. Industrial Inspection

- Detection of cracks, scratches and manufacturing defects.

5. Document Processing

- Removal of noise and enhancement of characters.

6. Surveillance Systems

- Enhancement of images captured under poor conditions.

7. Satellite Images

- Enhancement of geographical features and boundaries.

8. Object Detection

- Edge detection helps identify object boundaries.

Conclusion:

Spatial filtering is an important computer vision technique.

Low-pass filters are useful for noise reduction and smoothing,

while high-pass filters are useful for edge and feature detection.

The best filter depends on the image characteristics and the

requirements of the particular computer vision application.

""")

Save Results

```
# ------------------------------------------------------------


cv2.imwrite("mean_filter.jpg", mean_filter)

cv2.imwrite("gaussian_filter.jpg", gaussian_filter)

cv2.imwrite("median_filter.jpg", median_filter)

cv2.imwrite("laplacian_filter.jpg", laplacian)

cv2.imwrite("sobel_filter.jpg", sobel)

cv2.imwrite("canny_edges.jpg", canny)


print("\nFiltered images have been saved in the current folder.")
```

Sobel X - Horizontal Edges



Sobel Y - Vertical Edges



Combined Sobel



Laplacian Filtering



Average / Mean Filter



Median Filter



Gaussian Blur



Original Color Image



Original Grayscale Image



#Question And Answer

1. What is spatial filtering? How is it used in digital image processing?

Anwer - Spatial filtering modifies pixels using their neighboring pixels and a filter/kernel. It is used for noise removal, smoothing, sharpening, and edge detection.

2. Differentiate between Low-Pass Filters and High-Pass Filters with suitable examples.

Answer-Low-Pass Filter: Removes high-frequency details and smooths the image.
Example: Gaussian filter. High-Pass Filter: Enhances high-frequency details such as edges.
Example: Laplacian filter.

3. Compare Average Filter, Gaussian Filter, and Median Filter based on their working principles and applications.

Average Filter: Replaces a pixel with the average of its neighbors; used for smoothing.

Gaussian Filter: Uses weighted averaging; used for noise reduction and smoothing.

Median Filter: Replaces a pixel with the median of its neighbors; mainly used for salt-and-pepper noise removal.

4. Why is the Median Filter particularly effective for removing salt-and-pepper noise?

Answer: It removes extreme pixel values while preserving the middle value, so it removes black and white noise without significantly blurring edges.

5. Explain the role of convolution kernels in spatial filtering.

Answer: A convolution kernel is a small matrix that moves over an image and performs calculations on neighboring pixels. Different kernels produce effects such as smoothing, sharpening, and edge detection.

6. What is the purpose of the Sobel and Laplacian operators in edge detection?

Answer:

- Sobel Operator: Detects edges using intensity gradients in horizontal and vertical directions.
- Laplacian Operator: Detects edges using the second derivative of image intensity.

7. Why are filtering operations considered an essential preprocessing step in computer vision?

Answer: Filtering reduces noise, unwanted details, and variations, while enhancing important features. This improves the accuracy of later tasks such as segmentation, detection, and recognition.

8. Discuss the trade-off between image smoothing and edge preservation during filtering.

Answer: Strong smoothing removes more noise but can blur edges and details. Weak smoothing preserves edges but leaves more noise. Therefore, a suitable filter must balance noise reduction and edge preservation.

9. Mention four real-world applications where spatial filtering techniques are widely used.

Answer:

1. Medical imaging
2. Surveillance systems
3. Autonomous vehicles
4. Satellite/remote sensing images

10. Compare spatial domain filtering with frequency domain filtering in terms of implementation and practical applications.

Answer:

- Spatial Domain Filtering: Works directly on image pixels using kernels; commonly used for smoothing, sharpening, and edge detection.
- Frequency Domain Filtering: Converts the image into frequency components before filtering; useful for periodic noise removal and frequency-based image enhancement.